

IHK Datenstrukturen – Fragen und Antworten (2019–2025, KeepTogether)

Winter 2019/2020 – Array (Monatsumsätze) – Aufgabe

Deklarieren Sie ein Array `umsatz[12]` zur Speicherung der Monatsumsätze. Implementieren Sie eine Prozedur `addUmsatz(monat: Integer, betrag: Double)`, die den Umsatz zum jeweiligen Monat addiert.

Antwort (Pseudocode):

Array umsatz[12] von Double

```
Prozedur addUmsatz(monat: Integer, betrag: Double)
    umsatz[monat] = umsatz[monat] + betrag
end Prozedur
```

Sommer 2020 – Liste (Kundenverwaltung) – Aufgabe

Deklarieren Sie eine Liste `kunden: List`. Implementieren Sie `addKunde(k: Kunde)` und `removeKunde(nr: Integer)` (entfernt den ersten Kunden mit passender Nummer).

Antwort (Pseudocode):

Liste kunden

```
Prozedur addKunde(k: Kunde)
    kunden.add(k)
end Prozedur
```

```
Prozedur removeKunde(nr: Integer)
    for i = 0 bis kunden.size()-1
        if kunden.get(i).getKundennummer() == nr dann
            kunden.remove(i)
            return
        end if
    end for
end Prozedur
```

Winter 2020/2021 – Stack (LIFO) – Aufgabe

Implementieren Sie `push(x: Integer)` und `pop(): Integer` für einen Stack `s`.

Antwort (Pseudocode):

Stack `s`

```
Prozedur push(x: Integer)
    s.addTop(x)
end Prozedur
```

```
Funktion pop() → Integer
    if s.isEmpty() dann
        return -1
    end if
    return s.removeTop()
end Funktion
```

Sommer 2021 – Queue (FIFO) – Aufgabe

Implementieren Sie `enqueue(x: String)` und `dequeue(): String` für eine Warteschlange `q`.

Antwort (Pseudocode):

Queue `q`

```
Prozedur enqueue(x: String)
    q.addLast(x)
end Prozedur
```

```
Funktion dequeue() → String
    if q.isEmpty() dann
        return null
    end if
    return q.removeFirst()
end Funktion
```

Winter 2021/2022 – Assoziatives Array / Map – Aufgabe

Verwenden Sie eine Map `noteNachName: Map`. Implementieren Sie `setzeNote(name: String, note: Double)` und `holeNote(name: String): Double` (liefert -1.0, wenn nicht vorhanden).

Antwort (Pseudocode):

```
Map noteNachName // Schlüssel: String, Wert: Double
```

```
Prozedur setzeNote(name: String, note: Double)
    noteNachName.put(name, note)
end Prozedur
```

```
Funktion holeNote(name: String) → Double
    if noteNachName.containsKey(name) dann
        return noteNachName.get(name)
    else
        return -1.0
    end if
end Funktion
```

Sommer 2022 – 2D-Array (Tabelle) – Aufgabe

Implementieren Sie `printTabelle(tabelle: int[][])` , die alle Einträge zeilenweise ausgibt.

Antwort (Pseudocode):

```
Prozedur printTabelle(tabelle: int[][] )
    for i = 0 bis Länge von tabelle - 1
        for j = 0 bis Länge von tabelle[i] - 1
            print(tabelle[i][j])
        end for
        print("\n")
    end for
end Prozedur
```

Winter 2022/2023 – Dynamisches Array (Resize) – Aufgabe

Implementieren Sie eine vereinfachte dynamische Liste über ein Array `arr` mit `add(x: Integer)`, die bei voller Kapazität verdoppelt.

Antwort (Pseudocode):

Array arr von Integer, kapazität = 4, größe = 0

```
Prozedur add(x: Integer)
  if größe == kapazität dann
    newArr = neues Array von Integer der Länge kapazität * 2
    for i = 0 bis größe - 1
      newArr[i] = arr[i]
    end for
    arr = newArr
    kapazität = kapazität * 2
  end if
  arr[größe] = x
  größe = größe + 1
end Prozedur
```

Sommer 2023 – Verkettete Liste (einfach) – Aufgabe

Definieren Sie einen Knoten `Node(value: Integer, next: Node)` und implementieren Sie `prepend(x: Integer)` für eine einfach verkettete Liste.

Antwort (Pseudocode):

```
Klasse Node
  value: Integer
  next: Node
end Klasse
```

head: Node = null

```
Prozedur prepend(x: Integer)
  neu = Node()
  neu.value = x
  neu.next = head
  head = neu
end Prozedur
```

Winter 2023/2024 – Zirkuläre Queue (Ringpuffer) – Aufgabe

Implementieren Sie eine Ringpuffer-Queue mit Array `buf`, Indizes `front`, `rear`, Größe `n`. Methoden `enqueue(x)` und `dequeue()`.

Antwort (Pseudocode):

```
Array buf[n] von Integer
front = 0
rear = 0
anzahl = 0
```

```
Prozedur enqueue(x: Integer)
    if anzahl == n dann
        // voll
        return
    end if
    buf[rear] = x
    rear = (rear + 1) % n
    anzahl = anzahl + 1
end Prozedur
```

```
Funktion dequeue() → Integer
    if anzahl == 0 dann
        return -1
    end if
    x = buf[front]
    front = (front + 1) % n
    anzahl = anzahl - 1
    return x
end Funktion
```

Sommer 2024 – Set (Mengen, ohne Duplikate) – Aufgabe

Verwenden Sie ein Set `teilnehmerNamen: Set`. Implementieren Sie `addName(n: String)` und `istTeilnehmer(n: String): Boolean`.

Antwort (Pseudocode):

```
Set teilnehmerNamen
```

```
Prozedur addName(n: String)
    teilnehmerNamen.add(n)    // fügt nur hinzu, wenn noch nicht enthalte
end Prozedur
```

```
Funktion istTeilnehmer(n: String) → Boolean
    return teilnehmerNamen.contains(n)
end Funktion
```

Winter 2024/2025 – Prioritätswarteschlange (Min-Heap) – Aufgabe

Implementieren Sie eine `insert(x: Integer)` und `extractMin(): Integer` für einen Min-Heap als Array `heap`.

Antwort (Pseudocode):

Array `heap` von Integer
`größe = 0`

Prozedur `insert(x: Integer)`

`größe = größe + 1`

`i = größe`

`heap[i] = x`

 while `i > 1` und `heap[i] < heap[i/2]`

 tausche `heap[i]` mit `heap[i/2]`

`i = i / 2`

 end while

end Prozedur

Funktion `extractMin() → Integer`

 if `größe == 0` dann

 return -1

 end if

`min = heap[1]`

`heap[1] = heap[größe]`

`größe = größe - 1`

`i = 1`

 while `2*i <= größe`

`k = 2*i`

 if `k+1 <= größe` und `heap[k+1] < heap[k]` dann

`k = k + 1`

 end if

 if `heap[i] <= heap[k]` dann

 break

 end if

 tausche `heap[i]` mit `heap[k]`

`i = k`

 end while

 return `min`

end Funktion